

Homework Assignment

React Context / Modern Routing

3-4 day mini-project

Topic

Project Workspace Portal

Build a small React application using **React + TypeScript + React Router Data APIs**.

This homework is designed for approximately **3-4 days of work**.

The goal is to show that modern routing is not only page switching. In this assignment, the router should own route boundaries, data loading, mutations, navigation state, and URL-driven state. React Context should be used for **app-wide client state**, not as a replacement for route data.

Main learning goals

By the end of the homework, the solution should demonstrate:

- nested route architecture
- shared layout with <Outlet>
- route loader functions
- route action functions
- useFetcher for non-navigating mutations
- useNavigation for pending UI
- URL-based filtering through search params
- React Context for global client state such as theme or UI preferences
- clear separation between **Context**, **route data**, **URL state**, and **local component state**

Project idea

Build a small workspace application where a user can:

- open a dashboard page
- browse a projects list
- filter projects through the URL
- open project details by route param
- create a new project
- update some project data inline without leaving the page
- change an application-wide preference through React Context

You may rename the entities if you want, for example:

- projects -> tasks
- projects -> notes
- workspace -> team board
- workspace -> client portal

However, the route architecture and state responsibilities must stay the same.

Required route tree

Use a structure close to this:

```
/
|- index                dashboard
|- projects             list page
|- projects/:projectId  details page
`- settings             context-based page
```

You may add more routes if you want, but this minimum route tree is required.

Required functionality

1. Root layout

The root route must include:

- shared page shell
- navigation links
- `<Outlet>`
- a root loader
- visible global pending state using `useNavigation`

The root loader should return shared shell-level data. Examples:

- current user
- workspace summary
- counters for dashboard cards
- current organization name

2. Dashboard page

The dashboard page must be a real nested route.

It may be simple, but it should consume real route data. A good direction is to read root loader data through `useRouteLoaderData` instead of fetching the same information again.

3. Projects list page

The projects route must include:

- a loader
- reading search params from `request.url`
- URL-driven filtering
- a list view
- a route action for creating a new project

Minimum filter requirements:

- text query from search params
- one additional filter from search params, for example status

The filters must survive page reload because they live in the URL.

4. Project details page

The details page must include:

- a dynamic route param such as `:projectId`
- a route loader

- at least one useFetcher interaction that updates data without full navigation

Good examples of inline mutation:

- toggle favorite
- archive/unarchive item
- toggle priority
- mark as completed
- pin/unpin project

5. Settings page

The settings page must consume a **React Context** value and update it.

Recommended directions:

- theme switcher
- density mode
- language mock
- UI preferences panel

Context requirements

You must create at least one real context.

Recommended options:

- ThemeContext
- UiPreferencesContext
- ShellPreferencesContext

The context must include:

- a provider component
- a custom hook such as useTheme()
- at least one update function

Example direction:

```
type Theme = 'light' | 'dark';

type ThemeContextValue = {
  theme: Theme;
  toggleTheme(): void;
};
```

Important constraints:

- Context must be used for **client-owned app-wide state**
- Context must **not** be used to store loader data
- Context must **not** replace URL search params

If your setup supports the modern React 19 provider syntax, you may use it. If not, Context.Provider is also acceptable.

Routing requirements

Your solution must use a modern data-router setup.

Required APIs:

- createBrowserRouter
- RouterProvider

- nested routes
- <Outlet>
- loader
- action
- useNavigation
- useFetcher

Also required:

- route configuration must live outside React rendering
- route files must have clear boundaries
- the application architecture must be built around the route tree

Do not make the main architecture based only on:

- BrowserRouter
- <Routes>
- <Route>

Those APIs are not the main target of this homework.

Data loading requirements

Your application must demonstrate all of the following:

1. **Root loader** for shared shell data
2. **Leaf loader** for page-specific data
3. **URL-driven loader** that reads search params from `request.url`
4. **Route action** for form-based mutation
5. **Fetcher-based mutation** without route navigation

Example direction for a list route:

```
export async function loader({ request }: { request: Request }) {
  const url = new URL(request.url);
  const q = url.searchParams.get('q') ?? '';
  const status = url.searchParams.get('status') ?? 'all';

  return getProjects({ q, status });
}
```

Pending UI requirements

The app must visibly react to asynchronous router workflows.

Required:

- global pending navigation indicator via useNavigation
- local pending state for a fetcher interaction
- visible pending state for a form submission or route action

Error handling requirements

The application must include route-level error handling.

Accepted options:

- errorElement
- route error boundary

A blank page on load failure is not acceptable.

TypeScript requirements

- `strict: true` in `tsconfig.json`
- no `any`
- no disabled type checking
- clear types for loader data
- clear types for context values
- clear types for mutation payloads where appropriate

Architecture expectations

Do not put the whole app into one file.

A simple and readable structure is expected. Example:

```
src/  
  api/  
    fake-api.ts  
  components/  
  context/  
    theme-context.tsx  
  routes/  
    root.tsx  
    dashboard.tsx  
    projects.tsx  
    project-details.tsx  
    settings.tsx  
  router.tsx  
  main.tsx
```

You may choose another structure if it is consistent and easy to review.

Restrictions

Please keep the focus on built-in React and router APIs.

Do not use third-party global state libraries such as:

- Redux
- Zustand
- MobX
- Recoil
- Jotai

Small UI libraries are acceptable if they do not replace the main state and routing logic.

What may be simplified

- the UI may be simple
- CSS is not a grading focus
- you may use a fake API with `setTimeout`
- persistence may be in memory only
- authentication may be mocked or omitted

The main focus is:

- correct routing mental model
- route-driven data loading

- route-driven mutations
- URL-based state where appropriate
- proper Context responsibility
- clear project structure

Final submission

Please submit:

1. source code of the project
2. README.md with startup instructions
3. a short section called Architecture Notes
4. a short section called Used React Router APIs
5. a short section called Used Context Pattern
6. optional screenshot or short GIF

Acceptance criteria

The assignment is considered complete if:

- the project starts without manual fixes after installing dependencies
- TypeScript compiles without errors
- the app uses `createBrowserRouter` and `RouterProvider`
- the root layout renders child routes through `<Outlet>`
- the app has real nested routes
- the root route has a real loader
- at least one child route has a real loader
- at least one route reads filters from URL search params
- at least one route has an action
- at least one interaction uses `useFetcher`
- global pending UI works
- local pending UI works for a fetcher or form action
- a settings page reads and updates Context state
- route-level error handling exists
- the code clearly separates Context, route data, URL state, and local UI state

Recommended 4-day plan

Day 1

- scaffold the project
- define the route tree
- create the root layout
- implement the context provider and custom hook
- wire `RouterProvider`

Day 2

- add the root loader
- implement the dashboard route
- implement the projects route loader
- connect the projects list to URL search params

Day 3

- add the projects route action
- implement the create form
- implement global pending UI with `useNavigation`
- implement the project details loader

Day 4

- add one `useFetcher` interaction
- add route-level error handling
- review the separation between Context and route data
- polish naming and structure
- write the README and final notes

Bonus tasks

Choose one or more if you want to extend the project:

1. Add a protected route with loader-based redirect.
2. Persist theme in `localStorage`.
3. Add breadcrumbs using route metadata.
4. Add optimistic UI for one fetcher interaction.
5. Add one extra nested route under project details.

Suggested official references

- React Router documentation
- `createBrowserRouter`
- `RouterProvider`
- loader and action
- `useFetcher`
- `useNavigation`
- `React createContext`
- `React useContext`